How to Test This Project

Project: Auditable AI Agents
Everything below runs on your own computer. No cloud account or API key is needed.

---

1. One-time setup

```
cd /Users/mani-2209/AuditableAgentsProject
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
npm install
npx hardhat compile
```

For the real-LLM tests and demo only:

```
brew install ollama
ollama serve &            # or: brew services start ollama
ollama pull qwen3:8b      # about 5 GB, one time
```

---

2. Run every test

```
# Python tests (79)
python -m pytest -q

# Smart contract tests on a local chain (18)
npx hardhat test
```

Expected: `79 passed` and `18 passing`. Three tests are marked `live` (the real local chain, the end-to-end demo, and the real model). They use real services and **skip themselves** if the service is missing, so a green run without Ollama still passes (with skips reported by `pytest -rs`).

Useful variations:

Goal	Command
Offline tests only (no chain, no LLM): 76 tests	python -m pytest -q -m "not live"
Live tests only: 3 tests	python -m pytest -q -m live -rs
One file	python -m pytest -q tests/test_audit.py
One test, verbose	python -m pytest -v tests/test_audit.py::test_edit_is_detected_and_located
Show why tests were skipped	python -m pytest -q -rs
Stop at the first failure	python -m pytest -x -q

---

3. What each test file proves

File	Area	What it proves
tests/test_audit.py	Core cryptography	SHA-256 and canonical JSON behave; the hash chain verifies when intact; edit, delete, insert and reorder are detected and located; a fully rewritten entry still breaks later links; Merkle proofs verify for trees of 1 to 100 entries and fail for wrong entries or roots
tests/test_vectors.py	Specification	The Merkle format matches a by-hand computation (leaf and node prefixes, odd node carried up); the stored cross-language vectors are exactly what Python produces now
tests/test_fuzz_tamper.py	Randomised attacks	7 attack types x 300 random logs each are always detected; 300 untouched logs are never flagged (no false alarms); edits are located at the edited entry; a consistent rewrite passes the hash chain but fails the on-chain roots
tests/test_tools.py	Agent tools	Calculator values and rejection of unsafe input; word count; capital lookup; task planning; the scripted model's behaviour; unknown tools are logged as errors, not crashes
tests/test_agent_and_audit.py	Agent + audit	Every agent step is logged; batch planning; the verifier finds edits, rewrites, truncation; entries not yet anchored are reported, not flagged
tests/test_logstore.py	Log files	Save and load round trip, unicode, nesting, empty log; editing the file on disk is detected
tests/test_verify_log.py	Command-line checker	scripts/verify_log.py exits 0 on an intact log, 1 on a tampered one, 2 with no argument
tests/test_anthropic_llm.py	Claude adapter	Tool loop round trip and replay of model output blocks, error flagging, refusal handling, request settings (uses a fake client, no network)
tests/test_ollama_llm.py	Local-model adapter	Message conversion, request settings, audit metadata (uses a fake HTTP call)
tests/test_ollama_live.py	Real local model	qwen3:8b really completes a task, calls the right tools, and produces a verifying log (needs Ollama)
tests/test_chain_integration.py	Python + chain	Anchoring on a real local chain: contiguous batches, idempotent, per-account isolation, verification against on-chain data (needs npx)
tests/test_demo_e2e.py	Whole demo	The scripted demo detects all three tamper scenarios and recovers when the original is restored (needs npx)
chain-tests/AuditAnchor.test.js	Smart contract	Storage, events, reverts, append-only batches, per-agent isolation, gas ceiling; every Python-computed Merkle proof verifies on-chain, so the two languages agree on hashing

---

4. See it work: the end-to-end demo

```
python scripts/run_demo.py                 # scripted stand-in model, offline
python scripts/run_demo.py --llm ollama    # real local model (needs Ollama)
```

What you should see:

```
agent answered: Results: 84; 5; Tokyo
12 audit entries saved to research/experiments/agent_logs/...
anchored batch 0: entries 0..4, root f4873fc421a9..., gas 93100
```

```
...
verify untouched log: INTACT
verify after editing entry 3: TAMPERING DETECTED
verify after consistent rewrite (only the chain can catch this): TAMPERING DETECTED
verify after deleting the last 2 entries: TAMPERING DETECTED
verify after restoring the original: INTACT
```

The roots and gas values differ slightly with the model; the INTACT / TAMPERING DETECTED pattern must not.

---

5. Try to break it yourself (manual tamper test)

1. Produce a log: `python scripts/run_demo.py --llm scripted`
2. Check it is intact (hash chain only, no blockchain needed):

```
python scripts/verify_log.py research/experiments/agent_logs/scripted_run.jsonl
```

Expected: `INTACT: 12 entries, hash chain verifies`

3. Make a copy and edit it by hand, for example change `Tokyo` to `Kyoto`:

```
cp research/experiments/agent_logs/scripted_run.jsonl /tmp/edited.jsonl
sed -i '' 's/Tokyo/Kyoto/' /tmp/edited.jsonl
python scripts/verify_log.py /tmp/edited.jsonl
```

Expected: `TAMPERING DETECTED at entry ...` and exit code 1 (`echo $?`).

4. Try deleting a line, swapping two lines, or editing a timestamp. Every one should be flagged.
5. For the harder case, a consistent rewrite that the hash chain alone cannot see, run the full demo: it performs that attack and shows the on-chain roots catching it.

---

6. Test the blockchain part by hand

```
npx hardhat node          # terminal 1: a local chain with 20 funded test accounts
npx hardhat test --network localhost   # terminal 2: same tests, against that running node
```

You will see the node log each transaction as the tests run.

---

7. Checks that matter for the thesis

Claim	Evidence
Every tampering type is detected	test_fuzz_tamper.py: 7 attack types x 300 trials, zero misses
No false alarms	300 untouched logs, zero flags
Python and Solidity agree	Every Python proof for trees up to 100 entries verifies on-chain
A rewritten-but-consistent log is still caught	test_consistent_rewrite_... and the demo
It works with a real model	test_ollama_live.py, run_demo.py --llm ollama

Not yet measured (planned in `docs/methodology/evaluation-plan.md`): gas versus batch size, logging overhead per agent step, verification time at 100,000 entries.

---

8. Troubleshooting

Symptom	**Fix**
ModuleNotFoundError: web3 or pytest	Activate the venv: source .venv/bin/activate, then pip install -r requirements.txt
Chain tests skipped	Run npx hardhat compile (creates artifacts/) and make sure npx is on PATH
hardhat node did not start in time	A previous node may be holding the port; the demo picks a free port, so check npx hardhat node runs by itself
Ollama tests skipped	Start the server (ollama serve) and confirm ollama list shows qwen3:8b
Ollama run gives a slightly different answer	Local models are not perfectly repeatable; the audit log of that run will still verify
Compiler download failed	Hardhat downloads solc on first compile and needs internet once